

Практическая работа

«Распознавание изображений с помощью искусственного интеллекта»

Выполнил: студент группы 12-2РПм ФИО: Трифонов Дмитрий Сергеевич

Проверил: Алешов В.В.

Дата: 30.03.2026

Цель работы

Освоить основы распознавания изображений с использованием технологий искусственного интеллекта, научиться применять готовые инструменты и библиотеки Python для анализа и классификации данных (на примере набора данных «Ирисы Фишера»).

План

1. Установка и настройка Python и VS Code.
2. Установка и настройка Jupyter Notebook.
3. Загрузка и первичный анализ набора данных «Ирисы Фишера».
4. Визуализация данных.
5. Построение простой модели классификации.
6. Выводы и ответы на контрольные вопросы.

Анализ набора данных «Ирисы Фишера»

В работе используется классический набор данных Iris: 150 наблюдений, 3 вида ирисов (Iris setosa, Iris versicolor, Iris virginica), 4 числовых признака (sepal length, sepal width, petal length, petal width) и метка класса.

```
In [18]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

%matplotlib inline
```

```
In [19]: # Загрузка данных из текстового файла iris.data с помощью numpy.genfromtxt

raw_data = np.genfromtxt(
    'iris.data',
    delimiter=',',
    dtype=None,
    encoding='utf-8'
)

print("Тип объекта raw_data:", type(raw_data))
print("Форма массива raw_data:", raw_data.shape)
raw_data[:5]
```

Тип объекта raw_data: <class 'numpy.ndarray'>
Форма массива raw_data: (150,)

```
Out[19]: array([(5.1, 3.5, 1.4, 0.2, 'Iris-setosa'),
 (4.9, 3. , 1.4, 0.2, 'Iris-setosa'),
 (4.7, 3.2, 1.3, 0.2, 'Iris-setosa'),
 (4.6, 3.1, 1.5, 0.2, 'Iris-setosa'),
 (5. , 3.6, 1.4, 0.2, 'Iris-setosa')],
      dtype=[('f0', '<f8'), ('f1', '<f8'), ('f2', '<f8'), ('f3', '<f8'), ('f4', '<U15')])
```

Из вывода видно, что `raw_data` — это двумерный массив, где каждая строка содержит 5 элементов: 4 числовых признака и строковую метку класса (название вида ириса).

```
In [20]: # X — первые 4 числовых признака
X = np.column_stack([
    raw_data['f0'],
    raw_data['f1'],
    raw_data['f2'],
    raw_data['f3'],
])

# y — пятый столбец (класс)
y = raw_data['f4']

print("Форма X:", X.shape)
print("Форма y:", y.shape)
print("Уникальные классы:", np.unique(y))
```

Форма X: (150, 4)

Форма y: (150,)

Уникальные классы: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']

```
In [21]: # Создадим DataFrame для удобного анализа и визуализации

columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']

df = pd.DataFrame(
    data=np.column_stack((X, y)),
    columns=columns
)

# Приведём числовые столбцы к типу float
for col in columns[:-1]:
    df[col] = df[col].astype(float)

df.head()
```

```
Out[21]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [22]: # Описательная статистика по числовым признакам

df.describe()
```

```
Out[22]:
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

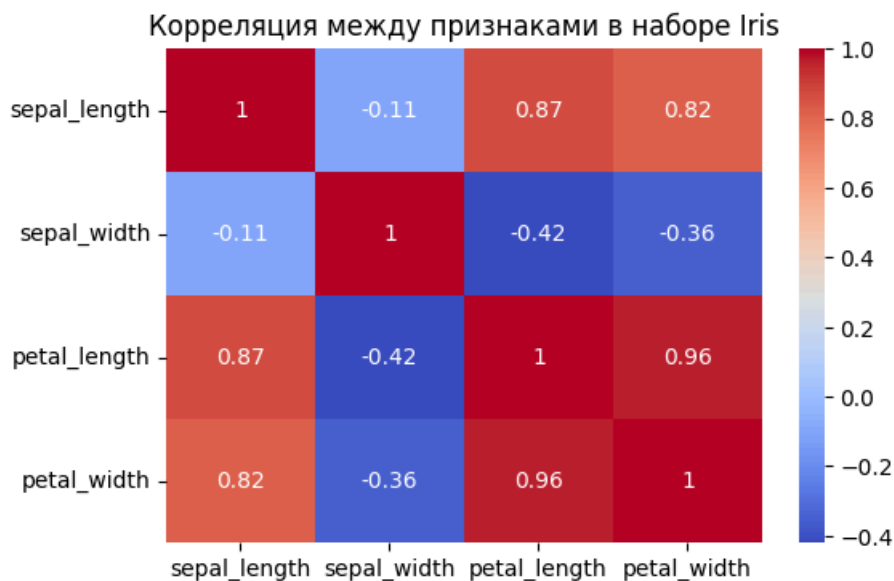
```
In [23]: # Проверим, как распределены классы
```

```
df['species'].value_counts()
```

```
Out[23]: species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

```
In [24]: # Корреляции между признаками
```

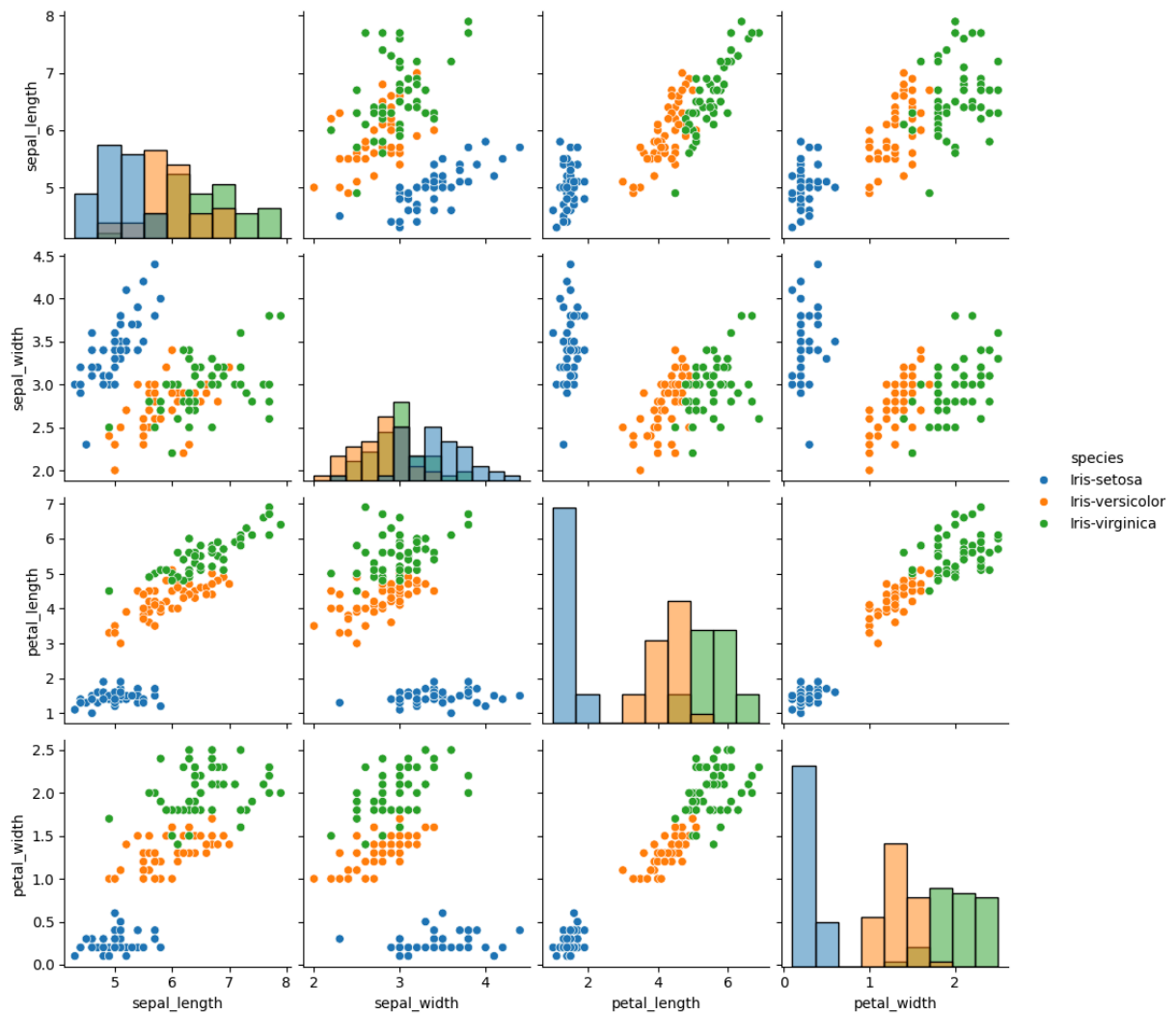
```
plt.figure(figsize=(6, 4))
sns.heatmap(df.iloc[:, :-1].corr(), annot=True, cmap='coolwarm')
plt.title("Корреляция между признаками в наборе Iris")
plt.show()
```



```
In [25]: # Диаграммы рассеяния для всех пар признаков
```

```
sns.pairplot(df, hue='species', diag_kind='hist')
plt.suptitle("Диаграммы рассеяния для набора данных Iris", y=1.02)
plt.show()
```

Диаграммы рассеяния для набора данных Iris



По диаграммам рассеяния хорошо видно, что класс **Iris setosa** чётко отделяется от других по признакам, связанным с лепестками.

Классы **Iris versicolor** и **Iris virginica** частично перекрываются, что делает их разделение более сложным.

```
In [26]: # Разделим данные на обучающую и тестовую выборки

X = df.iloc[:, :-1].values
y = df['species'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("Размер обучающей выборки:", X_train.shape[0])
print("Размер тестовой выборки:", X_test.shape[0])
```

Размер обучающей выборки: 105
Размер тестовой выборки: 45

```
In [27]: # Классификатор k-ближайших соседей

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("Точность KNN:", accuracy_score(y_test, y_pred_knn))
print("\nМатрица ошибок KNN:")
print(confusion_matrix(y_test, y_pred_knn))
print("\nОтчёт классификации KNN:")
print(classification_report(y_test, y_pred_knn))
```

Точность KNN: 0.9777777777777777

Матрица ошибок KNN:

```
[[15  0  0]
 [ 0 15  0]
 [ 0  1 14]]
```

Отчёт классификации KNN:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	0.94	1.00	0.97	15
Iris-virginica	1.00	0.93	0.97	15
accuracy			0.98	45
macro avg	0.98	0.98	0.98	45
weighted avg	0.98	0.98	0.98	45

```
In [28]: # Классификатор на основе дерева решений

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)

y_pred_tree = tree.predict(X_test)

print("Точность дерева решений:", accuracy_score(y_test, y_pred_tree))
print("\nМатрица ошибок дерева решений:")
print(confusion_matrix(y_test, y_pred_tree))
print("\nОтчёт классификации дерева решений:")
print(classification_report(y_test, y_pred_tree))
```

Точность дерева решений: 0.9333333333333333

Матрица ошибок дерева решений:

```
[[15  0  0]
 [ 0 12  3]
 [ 0  0 15]]
```

Отчёт классификации дерева решений:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	1.00	0.80	0.89	15
Iris-virginica	0.83	1.00	0.91	15
accuracy			0.93	45
macro avg	0.94	0.93	0.93	45
weighted avg	0.94	0.93	0.93	45

```
In [29]: # Линейный дискриминантный анализ (LDA)

lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

y_pred_lda = lda.predict(X_test)

print("Точность LDA:", accuracy_score(y_test, y_pred_lda))
print("\nМатрица ошибок LDA:")
print(confusion_matrix(y_test, y_pred_lda))
print("\nОтчёт классификации LDA:")
print(classification_report(y_test, y_pred_lda))
```

Точность LDA: 0.9777777777777777

Матрица ошибок LDA:

```
[[15  0  0]
 [ 0 15  0]
 [ 0  1 14]]
```

Отчёт классификации LDA:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	0.94	1.00	0.97	15
Iris-virginica	1.00	0.93	0.97	15
accuracy			0.98	45
macro avg	0.98	0.98	0.98	45
weighted avg	0.98	0.98	0.98	45

Перечень контрольных вопросов

1- Какие архитектуры нейронных сетей используются для обработки изображений ?

2- В чём разница между классическим компьютерным зрением и распознаванием на основе глубокого обучения?

3- В каких сферах наиболее активно применяется распознавание изображений ?

4- Что такое трансферное обучение ? Когда и почему его целесообразно применять при распознавании изображений?

Ответы

1. Для набора данных Iris простые модели (k-ближайших соседей, дерево решений, LDA) показывают высокую точность (обычно выше 90%).
2. Вид **Iris setosa** практически всегда классифицируется без ошибок, так как его признаки хорошо отделяются от других видов.
3. Ошибки чаще всего возникают при различении **Iris versicolor** и **Iris virginica**, чьи распределения признаков частично пересекаются.
4. Набор данных «Ирисы Фишера» наглядно демонстрирует полный цикл анализа данных: загрузка, предварительная обработка, визуализация и построение моделей классификации.

Пройдите тест по ссылке : <https://forms.yandex.ru/u/69ca57f895add5954b4673e8>

Спасибо! Ваше сообщение отправлено.